

GROUND SLOW, MOVE FAST

DualVLN

A Dual-System Foundation Model for
Generalizable Vision-and-Language Navigation

What it is, how it works, and how it compares with **StreamVLN**

Wei, Wan, Peng, Yu, Yang, Feng, Cai, Zhu, Wang, Pang, Liu · arXiv:2512.08186 · December 2025

Checkpoint: InternRobotics/InternVLA-N1-DualVLN · Code: InternRobotics/InternNav

What this talk covers

- 1 **Background** VLN-CE, the task and its metrics
- 2 **The problem** why end-to-end VLM → discrete action hurts
- 3 **DualVLN** System 2, System 1, and latent plans
- 4 **Results** R2R / RxR Val-Unseen against prior work
- 5 **StreamVLN** slow-fast context, the model we run today
- 6 **Head-to-head** architecture, output, sensing, latency
- 7 **Our stack** one HTTP contract, one trajectory follower
- 8 **Benchmark** the controlled A/B we can now run in Isaac

Vision-and-Language Navigation in continuous environments

The task

Give a robot one free-form sentence — "walk past the reception desk and stop at the second door" — and it must reach the described goal in an environment it has never seen, using only its own camera stream.

VLN-CE replaces the old navigation-graph teleports with continuous control, so the agent has to actually drive: no oracle waypoints, no map, no pre-built topology.

THE FOUR METRICS ON EVERY SLIDE

NE Navigation Error — metres from the goal at stop. Lower is better.

OS Oracle Success — did the path ever pass within 3 m of the goal.

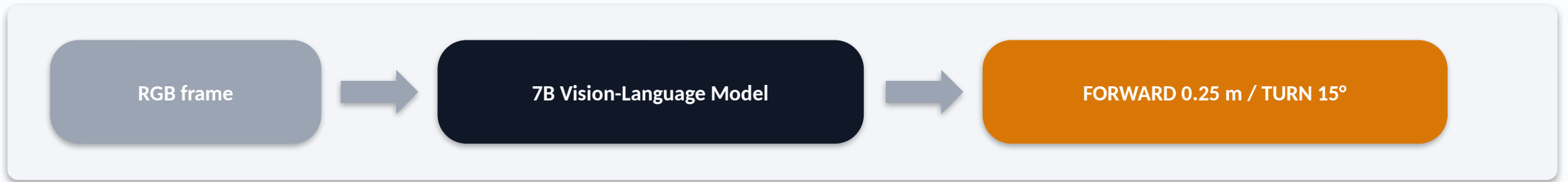
SR Success Rate — stopped within 3 m of the goal.

SPL Success weighted by Path Length — SR penalised for detours.

nDTW Path fidelity against the reference route (RxR).

Benchmarks: **R2R-CE** short, terse English instructions · **RxR-CE** long, multilingual, fine-grained instructions — the harder, more path-faithful split

Why one network mapping pixels → discrete actions falls short



- **The VLM sits in the control loop.** Every motion needs another billion-parameter forward pass, so the robot moves at the speed of the language model.
- **Motion is quantised.** A 0.25 m / 15° vocabulary cannot express a smooth arc; the robot stutters through stop-start bursts.
- **Reasoning horizon is short.** Predicting the next few actions is not the same as committing to a plan, so long instructions drift.
- **No room to react.** Between two VLM calls nothing is watching the world — a person stepping in front of the robot is seen late.

THE PAPER

Ground Slow, Move Fast

A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation

ARXIV

2512.08186 · 9 December 2025

AUTHORS

Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu,
Yuqiang Yang, Delin Feng, Wenzhe Cai,
Chenming Zhu,
Tai Wang, Jiangmiao Pang, Xihui Liu

MODEL

InternVLA-N1-DualVLN · 8B params · BF16

LICENCE

CC BY-NC-SA 4.0 (non-commercial)

CODE

github.com/InternRobotics/InternNav

Positioned as the dual-system release of the InternVLA-N1 navigation foundation model line.

Two systems, running at two different speeds

SYSTEM 2 “ground slow” ~2 Hz

VLM global planner

Qwen2.5-VL (7B), fully fine-tuned.

Reads the instruction and the current image and grounds the next sub-goal as a pixel goal — a point in the image — then emits a latent plan for System 1.

≈ **0.7 s** per trajectory-token inference with KV-cache reuse

SYSTEM 1 “move fast” ~30 Hz

Diffusion Transformer policy

384 hidden dim · 12 layers · 6 heads · DepthAnythingV2-Small ViT encoder.

Conditioned on the pixel goal and System 2's latent features, it outputs a short continuous trajectory rather than a discrete action.

32 trajectories in 0.03 s in parallel with TensorRT

Asynchronous by design. System 1 keeps driving while System 2 is still thinking — the robot never waits for the language model.

The key idea: latent plans instead of an explicit pixel goal

A pixel goal is ambiguous

"Go to that doorway" pinned to one image coordinate loses everything the planner knew but could not draw: which side to pass on, how far past it to stop, what the instruction said next.

A latent plan keeps it

System 2 hands System 1 a learned latent vector as well. It is a richer intermediate representation than a point, and it is trained end-to-end with the controller that consumes it.

TWO-STAGE CURRICULUM TRAINING

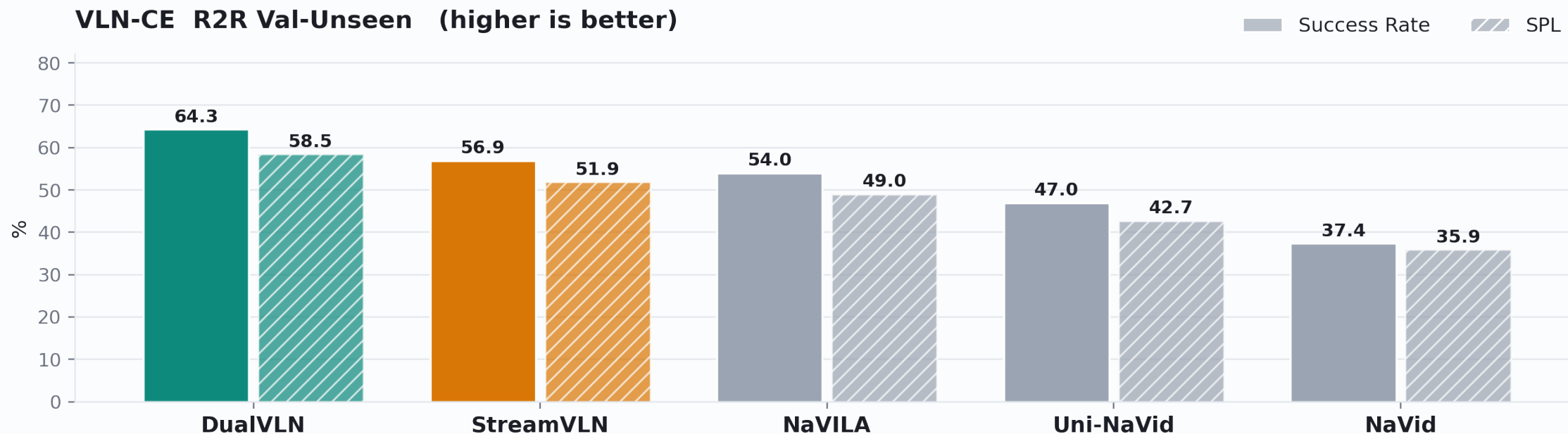
Stage 1 Pre-train both systems separately, supervised by / conditioned on explicit pixel goals.

Stage 2 Freeze System 2. Fine-tune the newly added latent plans jointly with System 1, asynchronously and end-to-end.

Result faster convergence and better end-to-end performance than the earlier explicit-goal InternVLA-N1 variants.

Read it as: the VLM stops being a controller and becomes a planner; a small fast policy becomes the controller.

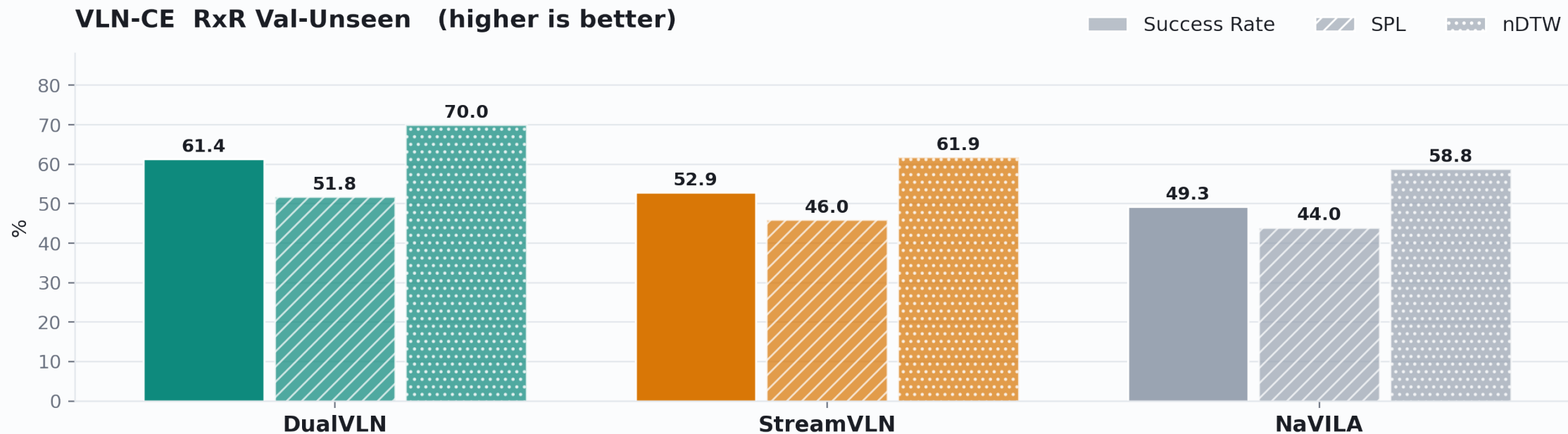
DualVLN leads every VLN-CE metric it reports



+7.4 SR and +6.6 SPL over StreamVLN on R2R Val-Unseen, and it beats NaVILA, Uni-NaVid and NaVid by a wider margin still.

All numbers as reported in the DualVLN paper's comparison table (arXiv 2512.08186). StreamVLN's own paper reports 56.4 SR / 50.2 SPL for its released checkpoint and 56.9 / 51.9 as its headline.

The gap widens on the harder, path-faithful split



+8.5 SR, +5.8 SPL, +8.1 nDTW over StreamVLN on RxR Val-Unseen. The nDTW jump matters most: it says DualVLN follows the described route, not just that it arrives.

Navigation error falls from 6.22 m to 4.58 m — the continuous controller stops closer to where the instruction actually pointed.

StreamVLN — the model we are running today

Streaming VLN via slow-fast context modelling

arXiv:2507.05240 · ICRA 2026 · InternRobotics — the same lab that released DualVLN.

StreamVLN attacks the same latency problem, but from the context side rather than the architecture side: keep one VLM, make its history cheap enough to run in real time.

HOW IT KEEPS UP

Fast dialogue context sliding window of 8 multi-turn dialogues drives responsive action generation.

Slow memory context 8×196 tokens of compressed history, pruned with a 3D-aware voxel strategy (−28 % to −32 % tokens).

KV-cache reuse over 99 % of prefill time removed; context size and cost stay bounded on long streams.

Backbone LLaVA-Video 7B / Qwen2-7B **Output** 4 discrete actions per call **Latency** 0.27 s per call **Training** 1 500 A100-hours

Head-to-head

	DualVLN	StreamVLN
Core idea	Split planning from control: two systems, two rates	Keep one VLM, make its context cheap
System 2 / backbone	Qwen2.5-VL 7B pixel-goal planner	LLaVA-Video 7B (Qwen2-7B)
System 1 / controller	Diffusion Transformer, 384-d, 12 layers	— none, the VLM is the controller
Model output	Continuous trajectory (waypoints)	4 discrete actions (0.25 m / 15°)
Interface	Latent plan + pixel goal	Autoregressive action tokens
Planner rate	≈ 2 Hz (0.7 s)	≈ 3.7 Hz (0.27 s per 4 actions)
Control rate	≈ 30 Hz (0.03 s for 32 trajectories)	Tied to the VLM call
Reacts mid-plan?	Yes — System 1 replans continuously	Only at the next VLM call
R2R Val-Unseen SR / SPL	64.3 / 58.5	56.9 / 51.9
RxR Val-Unseen SR / SPL	61.4 / 51.8	52.9 / 46.0

Rates are the figures each paper reports for its own deployment; they are not measured on the same hardware.

The difference that matters for a real robot

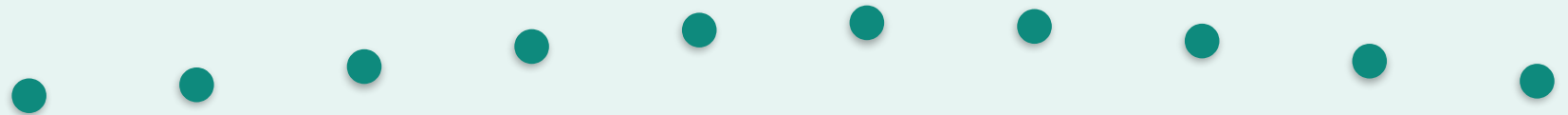
StreamVLN

one call →
4 quantised steps



DualVLN

one call →
a continuous path



Smoothness. A trajectory can be tracked by a normal velocity controller; an action list cannot be, without stop-start motion.

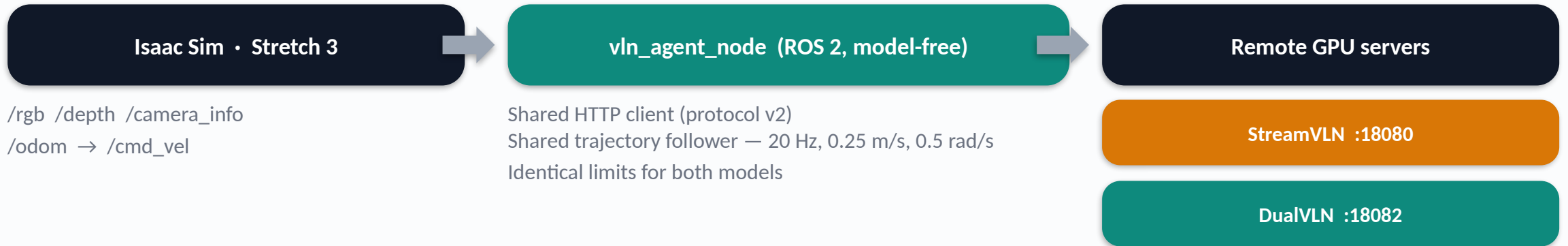
Reactivity. System 1 re-emits a path ~30× per second, so an obstacle appearing mid-plan is handled before the planner even notices it.

Sensing, licensing, and the practical caveats

	DualVLN	StreamVLN
Policy input (paper)	RGB — model card states depth not required	RGB stream
Released deployment code	Takes paired RGB + metric depth + intrinsics; System 1 consumes depth	RGB JPEG only
Licence	CC BY-NC-SA 4.0 — non-commercial	See StreamVLN repo
Model size	8B (7B VLM + small DiT)	7B
Real-robot platforms	Turtlebot4, Unitree Go2, Unitree G1	Unitree Go2
Extra evaluation	Human Collision Rate for dynamic/social scenes	Hallway / bedroom / office scenarios

Watch this one. The paper headlines RGB-only, but the published real-world agent feeds depth into System 1. Our integration therefore sends synchronised depth + intrinsics to DualVLN and records the sensor difference in the benchmark metadata.

Both models behind one interface in our stack



SWITCHING IS ONE ARGUMENT

```
./run_vln_demo.sh backend:=streamvln
./run_vln_demo.sh backend:=dualvln
```

- No model code in any ROS image.** Inference stays in two ROS-free GPU containers; the robot side is an HTTP client plus a controller.
- StreamVLN actions become waypoints.** Its 0.25 m / 15° steps convert losslessly into the same trajectory the follower runs for DualVLN — so the controller is never the variable under test.

The controlled A/B we can now run

- **Same prompt, verified.** Prompt hashes must match across both models or the run is rejected.
- **Same reset.** Isaac /set_entity_state teleport, then five consecutive odometry samples within 5 cm / 3° before the clock starts.
- **Same controller.** 20 Hz, 0.25 m/s, 0.5 rad/s, 0.35 m lookahead — identical for both backends.
- **Order alternated.** Reps 1/3/5 StreamVLN first; reps 2/4 DualVLN first, so warm-up drift cannot favour either.

WHAT EVERY TRIAL RECORDS

SPL @ 1 m and 3 m success only if the model itself stops in radius

Executed path integrated from raw odometry after reset

Shortest path from a verified reference polyline, never straight-line distance

Rate breakdown camera Hz, model updates/s, client Hz, server FPS, System 1 / System 2 FPS, controller Hz

Per-step JSONL one row per /step with poses, output and timings

Still to do: measure and verify the real hospital-scene start poses, goals and reference polylines — the manifest currently holds a placeholder route.

TAKEAWAYS

What to remember

1

DualVLN separates thinking from driving. A 7B VLM plans at ~2 Hz; a small diffusion policy drives at ~30 Hz. The robot stops waiting for the language model.

2

Latent plans are the contribution. A learned vector carries more of the planner's intent than a single pixel goal, and it is trained jointly with the controller that uses it.

3

It wins on every reported metric. +7.4 SR on R2R and +8.5 SR on RxR over StreamVLN, with a large nDTW gain — it follows the route, not just the destination.

4

StreamVLN is still the simpler system. One model, one context, RGB only, permissively usable. DualVLN is 8B, CC BY-NC-SA, and its deployment path wants depth.

5

We can now settle it empirically. Same prompt, same reset, same controller, five repetitions per route — on our own robot in our own scene.

References

Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation

Wei et al., arXiv:2512.08186, December 2025 · arxiv.org/abs/2512.08186

StreamVLN: Streaming Vision-and-Language Navigation via SlowFast Context Modeling

arXiv:2507.05240, ICRA 2026 · arxiv.org/abs/2507.05240

InternVLA-N1-DualVLN checkpoint

huggingface.co/InternRobotics/InternVLA-N1-DualVLN · CC BY-NC-SA 4.0

InternNav — inference code, evaluation protocol, Gradio demo

github.com/InternRobotics/InternNav · pinned at 7a5c624

StreamVLN implementation

github.com/InternRobotics/StreamVLN

Isaac Sim ROS 2 simulation control (/set_entity_state)

docs.isaacsim.omniverse.nvidia.com · 5.1.0 ros2_tutorials